

Informatik BW: Python Crash Course D

VU Informatik BW, WS24

Thorsten Ruprechter

- feedbackr.io (if-elif-else, lists)
- Loops (ff.)
- Functions (Basics)

Kurze Wiederholung: Sets und Dictionaries, Functions



fbr.io/infobw4

Zeit: 5 Minuten



- `input`
- `while`-Loops
- Functions (ff.)
 - Geschachtelte Funktionsaufrufe
 - Scoping: Globale und lokale Variablen
 - Call-by-Value vs. Call-by-Reference

5 User Input (0)

- Bisher: Programme ohne „Live-Interaktion“
- Mit Interaktion: Einlesen von User-Eingaben mittels `input()`

```
name = input('Tell me your name!')  
print(f'Oh, hi {name}!')
```

User Input (1): Nachbesserung und Korrektur

- String-Input = muss oft nachgebessert werden
 - Trimmen von Leerzeichen (z. B. durch `strip()`)
 - Umwandlung in andere Datentypen („Casting“)
 - Groß- und Kleinschreibung (`lower()`, `upper()`, `title()`)
 - etc.

```
name = input('Tell me your name! ' )  
corrected_name = name.strip()  
corrected_name = corrected_name.title()
```

Datentypen umwandeln: Casting

- Umwandeln von Wert in anderen Datentyp
 - `int()`, `str()`, `float()`, `bool()`...

- Beispiele

- `int('5')` $\Rightarrow$ `5`
- `float('5.23')` $\Rightarrow$ `5.23`
- `str(5)` $\Rightarrow$ `'5'`
- `int(True)` $\Rightarrow$ `1`
- `bool(0)` $\Rightarrow$ `False`

```
num1 = 123
num2 = '15'
num3 = num1 + num2
```

Output:

```
TypeError
  1 num1 = 123
  2 num2 = '15'
--> 3 num3 = num1 + num2
TypeError: unsupported operand
type(s) for +: 'int' and 'str'
```

Lösung:

```
num1 = 123
num2 = '15'
num3 = num1 + int(num2)
```

User Input (2): Beispiele

Siehe Code-Beispiele

while Loops (0): Basics

- Ausführen von Anweisungen bis Bedingung nicht mehr erfüllt
 - „Solange Bedingung erfüllt, führe Anweisungen aus“
- **SOLANGE** *Bedingung erfüllt:*
Führe Anweisungen aus
- **Python-Syntax:**

```
while condition:
    print( 'Running' )
```

```
x = 0
while x < 3:
    print(x)
    x = x + 1
print( 'done' )
```

while Loops (1): Kombination mit User-Input

- Möglichkeit: Ausführen bis spezieller Input
- Kontrolle durch Bedingungen oder „Flags“ (= Boolean-Variable)

```
user_input = ''  
while user_input != 'quit':  
    user_input = input('What are you saying?')  
    print(user_input)
```

while Loops (2): Endlosschleifen (Infinite Loops)

- Programm läuft unendlich
 - **while** True
- Abbruchbedingung durch **break**

```
while True:  
    user_input = input('What are you saying?')  
    print(user_input)  
    if user_input == 'quit':  
        break
```

while Loops und User Input Korrektur (3): Beispiele

Siehe Code-Beispiele

Functions (6): Arguments

- Argument-Typen bei Übergabe von Werten:
 - **Positional Arguments**
 - Identifiziert durch Position
 - **Keyword Arguments**
 - Identifiziert durch Name

```
def say_name(first, last):  
    print(f'Firstname is {first}, surname is {last}.')  
    print(f'I live in Austria.')  
say_name('Martina', last='Musterfrau')
```

Functions (6): Arguments

- Argument-Typen bei Übergabe von Werten:
 - **Positional Arguments**
 - Identifiziert durch Position
 - **Keyword Arguments**
 - Identifiziert durch Name
- **Default-Parameter:** Setzen von Werten für leergelassene Argumente

```
def say_name(first, last, country='Austria'):  
    print(f'Firstname is {first}, surname is {last}.')  
    print(f'I live in {country}.')  
say_name('Martina', last='Musterfrau')
```

Functions (7): Beispiele für valide Aufrufe

```
def say_name(first, last, country='Austria'):
    print(f'Firstname is {first}, surname is {last}.')
    print(f'I live in {country}.')
```

Passing arguments by position

```
say_name('Max', 'Muster')
```

```
say_name('Maria', 'Muster', 'Germany')
```

Passing arguments using keyword specification

```
say_name(first='Max', last='Muster')
```

```
say_name(country='Germany', first='Maria', last='Muster')
```

Mix of both

```
say_name('Maria', country='Germany', last='Muster')
```

Functions (8): Argument Errors

- Verwechseln von Argument-Reihenfolge
- Typo (Tippfehler) in Zuweisung der Keyword-Arguments
- Falsche Anzahl an Argumenten (TypeError)
- Positional Argument nach Keyword Argument
- Falscher Datentyp für Argument
- ...

Functions (9): Argument Errors (Beispiele)

```
def say_name(first, last, country='Austria'):
    print(f'Firstname is {first}, surname is {last}.')
    print(f'I live in {country}.')
```

```
say_name('Max')
```

```
# TypeError: say_name() missing 1 required positional argument: 'last_name'
```

```
say_name('Austria', 'Muster', 'Max')
```

```
# Kein Fehler in Ausführung, aber: "Logischer Fehler"
```

```
say_name(fs='Max', last='Muster')
```

```
# TypeError: say_name() got an unexpected keyword argument 'fs'
```

```
say_name(first='Max', 'Muster')
```

```
# SyntaxError: positional argument follows keyword argument
```

```
# There are many more possibilities for errors :-)
```

Funktions-Aufrufe schachteln: Argumente & Parameter

```
1 def get_len_list(elements):  
2     return f'List length is {len(elements)}'  
3  
4 def show_list_info(elements):  
5     print(f'List Elements are: {elements}')
```

Take a New Screenshot

```
6     print(get_len_list(elements))  
7  
8 def run_program():  
9     standard_elements = [3, '3', 'Three']  
10    show_list_info(standard_elements)  
11  
12 run_program()
```

Funktions-Aufrufe schachteln: Argumente & Parameter

```
1 def get_len_list(elements):
2     return f'List length is {len(elements)}'
3
4 def show_list_info(elements):
5     print(f'List Elements are: {elements}')
6     print(get_len_list(elements))
7
8 def run_program():
9     standard_elements = [3, '3', 'Three']
10    show_list_info(standard_elements)
11
12 run_program()
```

elements = elements

elements = standard_elements

List Elements are: [3, '3', 'Three']
List length is 3

Funktions-Aufrufe schachteln: Argumente & Parameter

```
1 def get_len_list(elements3):  
2     return f'List length is {len(elements3)}'  
3  
4 def show_list_info(elements2):  
5     print(f'List Elements are: {elements2}')6     print(get_len_list(elements2))  
7  
8 def run_program():  
9     elements1 = [3, '3', 'Three']  
10    show_list_info(elements1)  
11  
12 run_program()
```

→ List Elements are: [3, '3', 'Three']
List length is 3

Funktions-Aufrufe schachteln: Argumente & Parameter

```
1 def get_len_list(elements):
2     return f'List length is {len(elements)}'
3
4 def show_list_info(elements):
5     print(f'List Elements are: {elements}')
6     print(get_len_list(elements))
7
8 def run_program():
9     elements = [3, '3', 'Three']
10    show_list_info(elements)
11
12 run_program() →
```

List Elements are: [3, '3', 'Three']
List length is 3

Funktionen ff.: Funktions-Parameter mit selbem Namen

```
1 def get_len_list(elements):  
2     return f'List length is {len(elements)}'  
3  
4 def show_list_info(elements):  
5     print(f'List Elements are: {elements}')  
6     print(get_len_list(elements))  
7  
8 def run_program():  
9     elements = [3, '3', 'Three']  
10    show_list_info(elements)  
11  
12 run_program() →
```

List Elements are: [3, '3', 'Three']
List length is 3

Scoping (0): Globale und Lokale Variablen

- **Global**: Außerhalb von **Funktion** definiert
 - Lesbar in Funktionen
 - Unveränderbar innerhalb Funktion
 - **Verwendung meist nicht sinnvoll!**
- **Lokal**: Innerhalb von **Funktion** definiert
 - Nicht lesbar außerhalb von Funktion

```
def test_scope():  
    print('in1', y)  
    z = 5  
    print('in2', z)  
  
y = 4  
z = 3  
print('out1', z)  
test_scope()  
print('out2', z)
```

Scoping (0): Globale und Lokale Variablen

- **Global**: Außerhalb von **Funktion** definiert
 - Lesbar in Funktionen
 - Unveränderbar innerhalb Funktion
 - **Verwendung meist nicht sinnvoll!**
- **Lokal**: Innerhalb von **Funktion** definiert
 - Nicht lesbar außerhalb von Funktion
- Selbe Namen lokal und global möglich
 - Vermeiden um Problemen vorzubeugen!
- „Sichtbarkeitsbereich von Variablen“ = „Scope“

```
def test_scope():  
    print('in1', y)  
    z = 5  
    print('in2', z)  
  
y = 4  
z = 3  
print('out1', z)  
test_scope()  
print('out2', z)
```


Scoping (1): Globale und Lokale Variablen

- **Global**: Außerhalb von **Funktion** definiert
 - Lesbar in Funktionen
 - Unveränderbar innerhalb Funktion
 - **Verwendung meist nicht sinnvoll!**
- **Lokal**: Innerhalb von **Funktion** definiert
 - Nicht lesbar außerhalb von Funktion
- Selbe Namen lokal und global möglich
 - Vermeiden um Problemen vorzubeugen!
- „Sichtbarkeitsbereich von Variablen“ = „Scope“

ERROR!
(UnboundLocalError)

```
def test_scope():  
    print('in1', z)  
    z = 5  
    print('in2', z)  
  
y = 4  
z = 3  
print('out1', z)  
test_scope()  
print('out2', z)
```

Scoping (2): Globale und Lokale Variablen

- **Global**: Außerhalb von **Funktion** definiert
 - Lesbar in Funktionen
 - Unveränderbar innerhalb Funktion
 - **Verwendung meist nicht sinnvoll!**
- **Lokal**: Innerhalb von **Funktion** definiert
 - Nicht lesbar außerhalb von Funktion
- Selbe Namen lokal und global möglich
 - Vermeiden um Problemen vorzubeugen!
- „Sichtbarkeitsbereich von Variablen“ = „Scope“

```
def test_scope1():  
    y = x + 5  
    print('in1', y)
```

```
def test_scope2():  
    print('in2', x)  
    print('in2', y)
```

```
x = 3  
test_scope1()  
test_scope2()
```

Scoping (3): Globale und Lokale Variablen

- **Global**: Außerhalb von **Funktion** definiert
 - Lesbar in Funktionen
 - Unveränderbar innerhalb Funktion
 - **Verwendung meist nicht sinnvoll!**
- **Lokal**: Innerhalb von **Funktion** definiert
 - Nicht lesbar außerhalb von Funktion
- Selbe Namen lokal und global möglich
 - Vermeiden um Problemen vorzubeugen!
- „Sichtbarkeitsbereich von Variablen“ = „Scope“

ERROR!
(NameError)

```
def test_scope1():  
    y = x + 5  
    print('in1', y)
```

```
def test_scope2():  
    print('in2', x)  
    print('in2', y)
```

```
x = 3  
test_scope1()  
test_scope2()
```

Beispiele: Scoping (Globale/Lokale Variablen)

Siehe Code-Beispiele

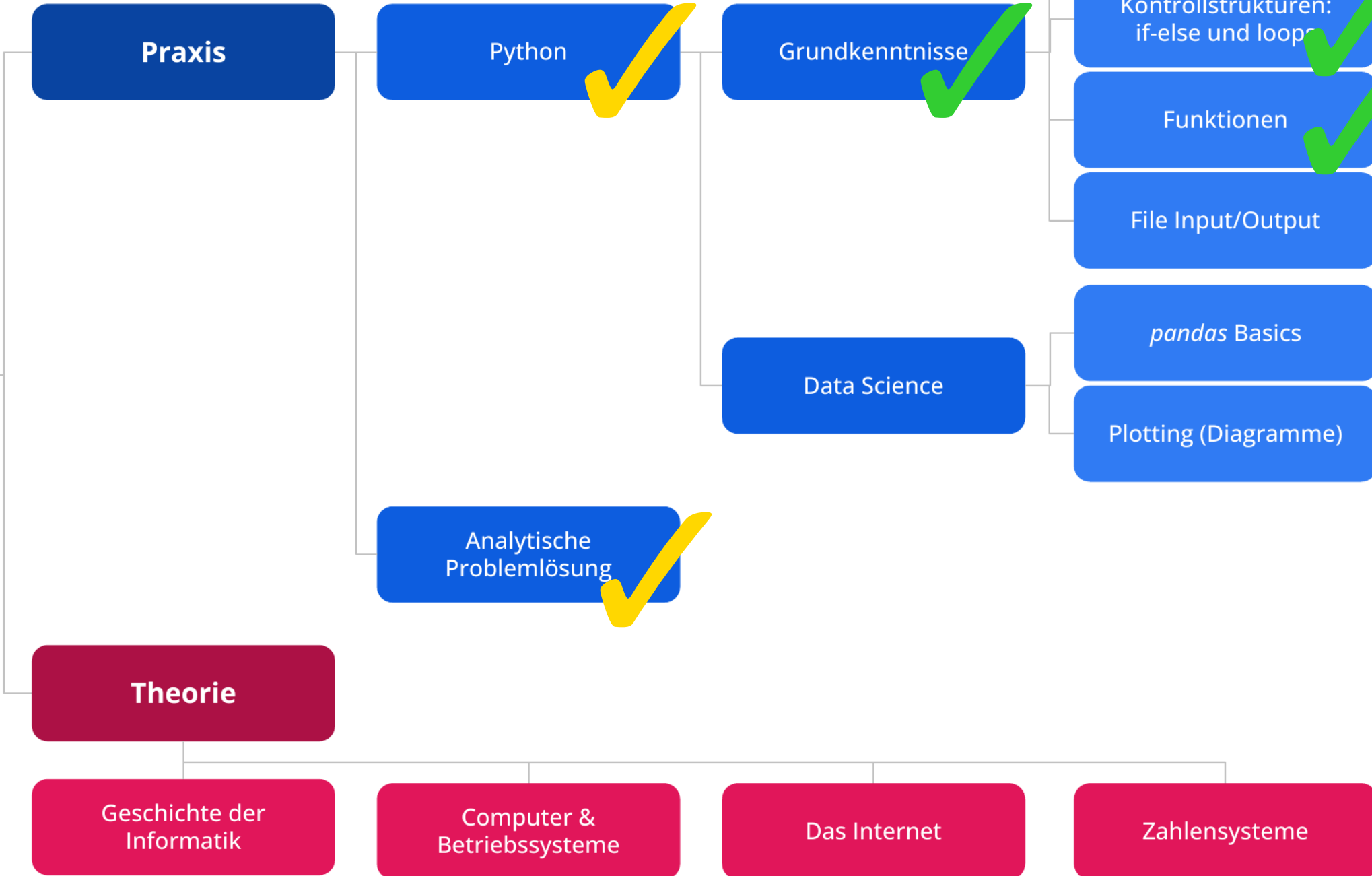
Kurze Wiederholung: Sets und Dictionaries, Functions



fbr.io/infobw4b

Zeit: 6 Minuten





Organisatorisches:

Verbleibende VU Outline (mit Vorbehalt, ohne Tutorien)

Typ	Datum	Details
VO	30. 10.	Asynchrone VO (Videos + feedbackr.io im TC)
VO	07. 11.	Asynchrone VO (Videos + feedbackr.io im TC)
UE	07. 11.	Deadline Ass1
AG	11. 11. – 13. 11.	Abgabegespräche Ass1 (AG1)
VO	14. 11.	Vorlesung HS G
VO	21. 11.	Vorlesung HS G
AG	27. 11.	Vorprogrammieren/Live-Programming
VO	28. 11.	Vorlesung HS G
UE	28. 11.	Ausgabe Ass2
VO	12. 12.	Vorlesung (Prüfungsvorbereitung)
UE	15. 12.	Deadline Ass2
AG	07. 01. – 08. 01.	Abgabegespräche Ass2 (AG2)
K	09. 01.	Klausur
NK	13. 01.	Nachklausur (bei negativer Hauptklausur)

- 30.10. & 07.11.: Keine LV im HS G
 - “Asynchrone” LVs
 - Inhalte im TeachCenter hochgestellt
 - feedbackr.io über Inhalte
 - Zählt zu Prüfungsstoff!

Hellgrau

= Übung

Dunkelgrau

= Vorlesung

Schwarz

= Anwesenheitspflicht (AG, VP, K)

Fett

= Deadlines Assignments

Informatik BW: Python Crash Course D

VU Informatik BW, WS24

Thorsten Ruprechter